



BLOCKCHAIN SECURITY GUIDE

Smart Contract Security

A field guide to identifying on-chain vulnerabilities, architectural patterns, and the review discipline required to protect decentralized protocols.

DOMAIN

AUDIENCE

LEVEL

EVM / Solidity

Devs & Auditors

Intro → Practitioner

PREPARED BY

BURAS Zakia

Date of creation • August 29, 2026

Reentrancy

Oracle

Flash Loans

Audits



Contents

00	Introduction
01	The EVM Execution Model
02	The Threat Landscape
03	The Vulnerability Catalogue
04	Securing a Smart Contract
05	Security Reviews & Audits
06	Incident Response
07	Pre-Deployment Checklist
08	Glossary & Resources

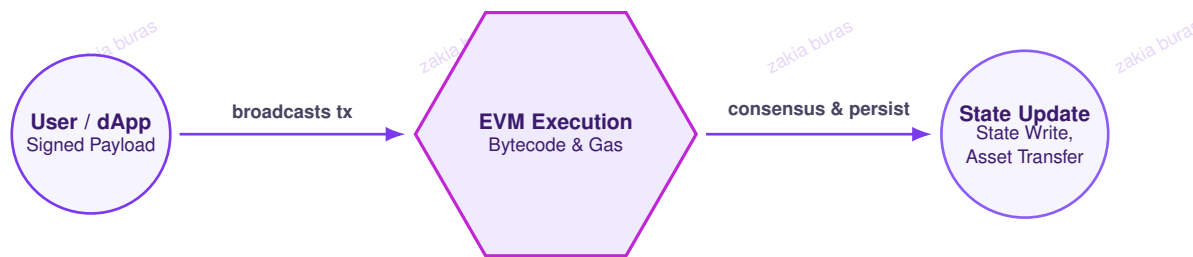
What is a smart contract?

A smart contract is a program stored on a blockchain that runs automatically when preset conditions are met, enforcing an agreement without banks, brokers, or any middleman. Its code, and every action it takes, is recorded on a public ledger that no single party controls.

This is what makes smart contracts important: they let people who don't trust each other transact and coordinate under rules that execute exactly as written, transparently and automatically.

They are the engine behind decentralized finance (DeFi), tokens and NFTs, DAOs, and on-chain marketplaces, autonomous programs that hold and move real value on their own.

0.1 Smart Contract Execution Flow



Schema 1. Smart contract execution and state transition workflow.

0.2 Why smart contract security matters

Because a smart contract holds real value and runs on a public, adversarial network, the very properties that make it "trustworthy" also make it "dangerous".

Once deployed, its code is typically **immutable** and a bug cannot be quietly patched. Its bytecode and state are **fully visible**, so attackers study it at leisure. And every transaction it permits is **irreversible**: a single overlooked line can be exploited by anyone, at any time, for the entire balance of the contract. Security is therefore not a feature you add later but it is the price of admission.

0.3 Numbers to remember I was shocked too

According to CertiK's H1 2026 security report, over \$1.3 billion was stolen on-chain in the first half of the year alone, despite audits. Most of that loss came not from broken code but from compromised keys and signers, the humans around the code rather than the code itself. Passing an audit is not the same as being secure. For detailed case files on individual incidents, see rekt.news.

01

The EVM Execution Model

Master the structural mechanics that define the attack surface, and vulnerability classes cease to be arbitrary anomalies they resolve into a predictable, systematic taxonomy.

1 The EVM Execution Model

Before studying how contracts break, you need the handful of EVM behaviours every attack relies on. Each one below is directly responsible for a real-world **disaster** you will meet in the next section.

1.1 Message calls: who is really calling you?

Every interaction is a *message call*, two variables tell you where it came from, and confusing them is a classic bug:

- `msg.sender` is the **immediate** caller: the contract or wallet that called you directly.
- `tx.origin` is the wallet that **started** the whole chain of calls, however deep it goes.

Let's say there is a person called **Zakia** :). **Zakia** calls a malicious contract, which then calls your contract. In this case, `msg.sender` is the malicious contract, while `tx.origin` is still **Zakia**. Authorizing based on `tx.origin` therefore trusts **Zakia**, even though *the attacker* is the one actually giving the orders.

1.1.1 Vulnerable Pattern vs. Secure Implementation

#VULNERABLE

```
require(tx.origin == owner);    // a phishing contract calling as Zakia: PASSES
```

authenticate the direct caller

```
require(msg.sender == owner);    // the malicious contract is not the owner: REVERTS
```

1.2 Function visibility and access control

Function visibility determines who can call a function. Using a broader visibility than necessary can unintentionally expose sensitive functionality.

- **external**: callable only from outside the contract.
- **public**: callable both internally and externally.
- **internal**: callable within the contract and its derived contracts.
- **private**: callable only within the defining contract.

Unintended external access

```
contract Vault {
    function _moveFunds(address to, uint amount) public {
        payable(to).transfer(amount);
    }
    function withdraw(uint amount) external {
        require(balances[msg.sender] >= amount);
        balances[msg.sender] -= amount;
        _moveFunds(msg.sender, amount);
    }
}
```

Because `_moveFunds()` is **public**, anyone can call it directly and bypass the checks in `withdraw()`. The safer approach is to use the narrowest visibility required:

Use the narrowest visibility required

```
function _moveFunds(address to, uint amount) internal { // DANGEROUS
    payable(to).transfer(amount);
}
```

1.3 Modifiers Are Only as Strong as Their Checks

Modifiers provide reusable access checks, but their security depends on the state they rely on. If that state can be manipulated by an unauthorized caller, the modifier can be bypassed.

Modifier relying on mutable state

```
modifier onlyAfterUnlock() {
    require(block.timestamp >= unlockTime, "locked");
    _;
}
function setUnlockTime(uint time) external {
    unlockTime = time; // Anyone can modify it
}
```

The check is valid, but `unlockTime` is not protected. An attacker can therefore manipulate the state that the modifier trusts.

Protect the state used by the modifier

```
modifier onlyOwner() {
    require(msg.sender == owner, "not owner");
    _;
}
modifier onlyAfterUnlock() {
    require(block.timestamp >= unlockTime, "locked");
    _;
}

function setUnlockTime(uint time) external onlyOwner {
    unlockTime = time;
}
```

Mitigation: Restrict all functions that modify security-critical state to authorized accounts, and validate the new value where necessary. A modifier is only reliable when the state it depends on cannot be manipulated by unauthorized callers.

1.4 Fallback Execution and External Code Invocation

When ETH is sent to a contract, its `receive()` or `fallback()` function may execute immediately. With `call`, this gives the recipient an opportunity to execute arbitrary code before the original function completes.

External Code Execution

```
receive() external payable {
    // Attacker-controlled code can execute here
}
```

This creates a potential re-entrancy vector: the recipient can call back into the sender before its state has been updated.

1. The contract initiates an ETH transfer.
2. The recipient's `receive()` function executes.
3. The recipient calls back into the original contract.
4. Unupdated state may allow the operation to be repeated.

Key point: This constitutes a **re-entrancy vulnerability**, while external code execution is unavoidable, strict adherence to state-management patterns mitigates the risk (see Section 2).

1.5 Value Transfer Mechanisms in Solidity

Solidity provides three primitives for transferring native **ETH**, differing in gas forwarding and failure handling.

1.5.1 Comparative Summary

Method	Gas warded	For-	Behavior and Architectural Impact	Verdict
transfer	Fixed (2,300)		Reverts on failure. The gas stipend can prevent many re-entrant calls, but may break smart-contract recipients after gas repricing.	Avoid
send	Fixed (2,300)		Returns <code>false</code> on failure instead of reverting, making unchecked failures possible.	Avoid
call	Configurable		Returns <code>(bool, bytes)</code> and supports arbitrary recipient logic. Requires explicit re-entrancy protection.	Recommended

1.5.2 Why `call` is the Standard

`call` provides greater compatibility with smart-contract recipients without relying on a fixed gas stipend. However, because it forwards execution to the recipient, re-entrancy must be explicitly prevented.

1.5.3 Code Comparison: Avoid vs. Recommended

#VULNERABLE

```
// AVOID: Fixed 2,300 gas stipend
msg.sender.transfer(amount);
```

Recommended: Explicit error handling

```
(bool success, ) = msg.sender.call{value: amount}("");
require(success, "Transfer failed");
```

Mitigation: use `call` for ETH transfers, check its return value, and protect against re-entrancy.

1.6 Storage, `delegatecall`, and Visibility

Smart contract state is stored in deterministic storage slots. Incorrect assumptions about storage layout and execution context can introduce severe architectural vulnerabilities.

1.6.1 `delegatecall`: Execution Context and Storage Layout

Unlike `call`, which executes code using the target's storage, `delegatecall` executes the target's code using the caller's storage and execution context.

Metric	<code>call</code>	<code>delegatecall</code>
Storage Context	Target contract storage	Caller contract storage (mutable)
Execution Context	Target contract (<code>msg.sender = Caller</code>)	Caller contract (<code>msg.sender</code> preserved)
Primary Use Case	Inter-contract interaction, asset transfer	Proxy patterns

The risk: storage collision

With `delegatecall`, the implementation writes to the caller's storage slots. If the layouts differ, a variable in the implementation can overwrite an unrelated variable in the proxy.

Incompatible storage layouts

```
contract Logic {
    address public owner;           // slot 0
    function setOwner(address a) external {
        owner = a;                 // writes slot 0
    }
}

contract Proxy {
    uint256 public total;           // slot 0

    function execute(address impl) external {
        impl.delegatecall(
            abi.encodeWithSignature("setOwner(address)", msg.sender)
        );
    }
}
```

Danger: `setOwner()` writes to slot 0. With `delegatecall`, this slot belongs to Proxy, causing `total` to be overwritten instead of `owner`.

Mitigation: Maintain compatible storage layouts and use established upgradeable-contract standards instead of manually managing proxy storage.

- **OpenZeppelin Upgradeable Contracts:** Use audited UUPS or transparent proxy implementations.
- **EIP-1967:** Use standardized storage slots for proxy-specific state.
- **Initializers:** Use initializer functions instead of constructors in upgradeable implementations.

References: [OpenZeppelin Upgradeable Contracts](#) [EIP-1967](#).

1.6.2 `private` does not mean secret

`private` only stops *other contracts* from reading a variable in Solidity. It is not encryption: the value still sits in a numbered storage slot, and anyone can read that slot straight off the chain.

THE PROBLEM: `private` is still readable

```
contract Game {
    uint256 private password = 42; // stored in slot 0
}

// Anyone can read that slot directly from:
// Foundry CLI:
bytes32 secret = vm.load(gameAddress, bytes32(uint256(0))); // returns 42

// Ethers.js:
const secret = await provider.getStorage(gameAddress, 0); // -> 42
```

The fix is **commit-reveal**: never put the secret on-chain. You hash it **off-chain** (locally) and submit only the **hash**; you reveal the underlying value later, allowing the contract to verify integrity.

The fix: commit the hash, reveal later

```
// OFF-CHAIN (local computation): hash = keccak256(answer, salt)
bytes32 public commitment;

function commit(bytes32 hash) external { // Stores the pre-computed hash
    commitment = hash;                  // Only the cryptographic fingerprint is saved
}

function reveal(uint256 answer, bytes32 salt) external {
    require(keccak256(abi.encodePacked(answer, salt)) == commitment, "Invalid reveal");
}
```


The rule: hiding data on-chain Everything on-chain is public visibility only prevents other contracts from reading it, not external observers. Select your approach based on required persistence:

- **Temporarily** (auctions, games): Publish a cryptographic **hash** and reveal later. Include a high-entropy salt to prevent brute-force attacks on low-entropy values.
- **Permanently** (credentials, sensitive keys): Keep data strictly **off-chain**; it must never touch public ledger state.
- **Verifiably** (zk validation): Prove knowledge of data without revealing it via **zero-knowledge proofs**.

1.6.3 initializer: Replacing Constructors in Upgradeable Contracts

Constructors run only when the implementation is deployed, so they cannot initialize the proxy's storage. Upgradeable contracts instead use an `initialize()` function called through `delegatecall`.

Because `initialize()` is a regular function, it must use the `initializer` modifier to prevent repeated initialization.

Initializer Pattern (Pattern-Agnostic)

```
contract MyLogic is Initializable {
    address public owner;

    function initialize(address _owner) public initializer {
        owner = _owner; // Replaces native constructor logic
    }
}
```

Critical Security Implications: Omitting the `initializer` modifier or failing to execute the initialization call immediately post-deployment allows malicious actors to front-run the call, initialize the contract themselves, and assume unauthorized ownership.

1.6.4 selfdestruct: Contract Removal and Associated Risks

The `selfdestruct(address payable recipient)` opcode removes a contract's code from the blockchain and forcefully transfers its remaining ETH balance to the specified recipient. Although historically deployed for cleanup routines or emergency kill switches, modern Ethereum architecture renders it a critical vulnerability vector.

Why selfdestruct Is Risky

```
function kill() public {
    selfdestruct(payable(owner));
}
```

Primary Security Risks:

- **Forced ETH Injection:** Bypasses `receive()/fallback()`, allowing attackers to force-feed funds and break assumptions tied to `address(this).balance`.
- **Dependency Collapse:** Eradicating a core `delegatecall` target permanently disables or destabilizes all dependent contracts.

Mitigation: Avoid `selfdestruct` entirely. Use audited pausing patterns (e.g., OpenZeppelin `Pausable`) for emergency halts and avoid strict balance checks.

1.6.5 fallback() vs. receive(): Which One Runs?

Solidity provides two special functions for handling ETH transfers and unmatched calls:

- `receive() external payable`: runs when ETH is sent with empty calldata.
- `fallback() external [payable]`: runs when no matching function exists, or when ETH is sent with calldata and no `receive()` is defined.

No receive() or payable fallback()

```
contract NoReceive {  
    // Plain ETH transfers revert.  
}
```

Security consideration: Both functions can execute attacker-controlled calls, so they should remain minimal and avoid unnecessary state changes or external calls.

Minimal receive() and fallback()

```
receive() external payable {  
    emit Received(msg.sender, msg.value);  
}  
fallback() external {  
    revert("Unknown function");  
}
```

1.6.6 Enforcing Rigor: Modifiers and Event Emission Discipline

Securing smart contracts is as much about code discipline as it is about avoiding dangerous opcodes. Two fundamental practices must never be treated as optional boilerplate:

- **Mandatory Access Control Modifiers:** Encapsulate access control and validation logic within well-tested, reusable modifiers (e.g., `onlyOwner`) to maintain a single source of truth and eliminate human error.

Access Control Modifier Pattern

```
address public owner;  
  
modifier onlyOwner() {  
    require(msg.sender == owner, "Unauthorized");  
    _; // Execution continues here if check passes  
}
```

- **Disciplined Event Emission:** Every critical state change, ownership transfer, or value movement should emit an indexed event. Without proper events, monitoring and post-mortem analysis become significantly harder.

Event Emission Discipline

```
// Explicit event definition with indexed parameters for tracking  
event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);  
  
function transferOwnership(address newOwner) external onlyOwner {  
    address oldOwner = owner;  
    owner = newOwner;  
  
    // Mandatory event emission on critical state changes  
    emit OwnershipTransferred(oldOwner, newOwner);  
}
```

1.7 Zero-Knowledge Proof (ZKP) Integration Vulnerabilities

ZKP integrations introduce cryptographic risks beyond standard Solidity vulnerabilities. Flaws in circuit design, proof verification, or trusted setup can compromise the entire system.

1.7.1 Trusted Setup Ceremony Flaws (Groth16)

Groth16 requires a trusted setup ceremony that generates public parameters and secret randomness known as “toxic waste.” This secret material must remain unknown and be securely destroyed.

- **Risk:** If the toxic waste is compromised, the soundness of the proving system can be undermined, potentially allowing forged proofs.

- **Mitigation:** Use a transparent, setup-free proving system such as a **STARK** when eliminating trusted-setup assumptions is a requirement.

1.7.2 Pairing and Curve Validation

Groth16 verifiers rely on elliptic-curve pairings. Incorrect validation of proof points can undermine verification.

- **Risk:** Missing curve or subgroup validation can allow malformed points to bypass the intended verification assumptions.
- **Mitigation:** Use audited verifier generators and established pairing implementations rather than writing custom cryptographic verification logic.

1.7.3 Non-Canonical Public Inputs

Public inputs are interpreted as elements of a finite field and must satisfy the verifier's field constraints.

Missing field-range validation

```
function verifyProof(
  uint256[2] calldata a,
  uint256[2][2] calldata b,
  uint256[2] calldata c,
  uint256[1] calldata input
) external view returns (bool) {
    // Missing canonical range validation
    return Pairing.pairingCheck(a, b, c, input);
}
```

Risk: Accepting values outside the expected field range can create ambiguity between different integer representations of the same field element.

Mitigation: Validate public inputs against the expected field modulus before verification.

Canonical public-input validation

```
require(input[0] < SCALAR_FIELD_MODULUS, "invalid field element");
return Pairing.pairingCheck(a, b, c, input);
```

1.7.4 Under-Constrained Circuits

ZKP vulnerabilities can originate in the circuit rather than the Solidity verifier. An under-constrained circuit fails to enforce all required relationships between its inputs and witness variables.

- **Risk:** Missing constraints can allow a prover to generate a valid proof for a statement that does not satisfy the intended security property.
- **Mitigation:** Review circuit constraints independently from the Solidity verifier and use dedicated circuit-analysis and formal-verification techniques where appropriate.

1.8 Gas Optimization Patterns

Gas optimization should reduce execution costs without compromising correctness or security. The following techniques cover common optimizations before resorting to low-level code.

1.8.1 Storage Packing

Solidity can pack multiple smaller state variables into a single 32-byte storage slot. Careful ordering can therefore reduce storage usage and gas costs.

Struct Packing

```
// Wasteful: uint256 separates the uint32 fields
struct Wasteful {
    uint32 a;
    uint256 c;
    uint32 b;
}

// Efficient: both uint32 fields share one slot
struct Efficient {
    uint256 c;
    uint32 a;
    uint32 b;
}
```

Rule: group variables that can share a storage slot and use types that safely fit the required range. Consider access patterns as well, since packing can increase the cost of updating individual fields.

1.8.2 Common Optimization Patterns

- **Use calldata for read-only external parameters.** This avoids unnecessary memory copies.
- **Cache repeated storage reads.** Store frequently accessed values in local variables instead of repeatedly reading from storage.
- **Use unchecked only when overflow is provably impossible.** This removes unnecessary arithmetic checks without sacrificing safety.
- **Prefer custom errors over long revert strings.** Custom errors generally reduce deployment and revert-data costs.
- **Order short-circuit conditions efficiently.** Place inexpensive or likely-to-fail conditions first in `&&` and `||` expressions.

#VULNERABLE

```
function processItems(uint[] memory items) external {
    for (uint i = 0; i < items.length; i++) {
        total += items[i];
        total += balances[msg.sender];
    }
}
```

Cache repeated reads

```
function processItems(uint[] calldata items) external {
    uint len = items.length;
    uint bal = balances[msg.sender];
    for (uint i = 0; i < len; i++) {
        total += items[i];
        total += bal;
    }
}
```

1.9 Funds Sent to a Broken Contract Do Not Return Automatically

ETH and tokens sent to a contract do not automatically return if the contract lacks a valid recovery path. If the contract accepts funds but its logic cannot transfer them out, those funds may become permanently inaccessible.

- **Reverted transaction:** Funds never leave the sender, so nothing becomes stuck.
- **Successful but incorrect accounting:** Funds remain in the contract, but may become inaccessible if no recovery mechanism exists.

- **Recovery:** Funds can only be recovered through functionality that was already deployed, such as a carefully restricted emergency mechanism.

Restricted emergency recovery

```
function emergencyWithdraw(
  address token,
  uint256 amount
) external onlyOwner {
  if (token == address(0)) {
    (bool success, ) =
      payable(owner()).call{value: amount}("");
    require(success, "ETH transfer failed");
  } else {
    IERC20(token).transfer(owner(), amount);
  }
  emit EmergencyWithdrawal(token, amount);
}
```

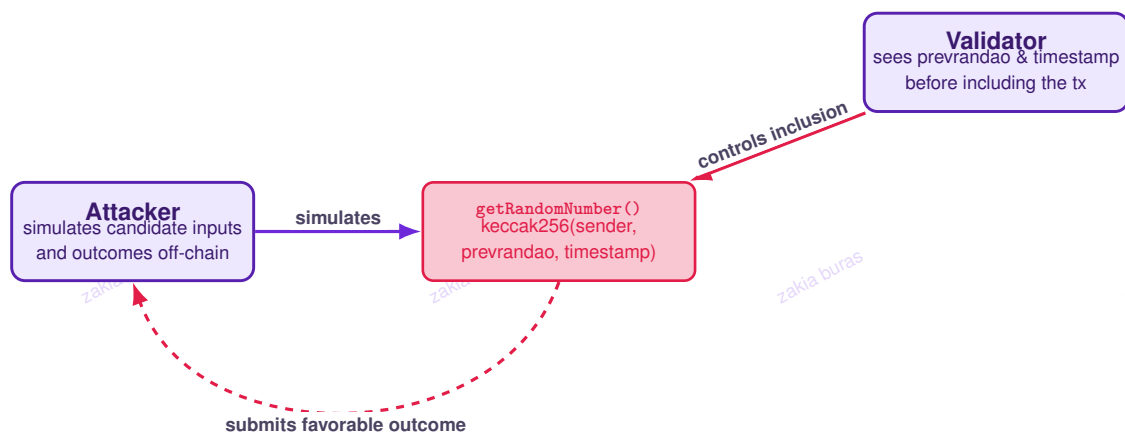
Security trade-off: Recovery functions introduce privileged control over funds. Restrict them with strong governance, such as a multisig and timelock, and define clearly which funds they may recover.

Security trade-off: Recovery functions introduce privileged control over funds. Restrict them with strong governance, such as a multisig and timelock, and define clearly which funds they may recover.

1.10 Why block.prevrando is Not Secure Randomness

After the Merge, Ethereum replaced block.difficulty with the PREVRANDAO value (EIP-4399), exposed in Solidity as block.prevrando. Although it provides stronger randomness than block.difficulty, it is not suitable for applications requiring unpredictable, manipulation-resistant randomness.

- **Predictable before execution:** The value is available as part of the block data, so participants can determine the result before the contract executes.
- **Validator influence:** A block proposer may have some ability to influence whether a transaction is included when the resulting outcome is unfavorable.
- **Caller manipulation:** If the caller's address is included in the randomness calculation, an attacker may simulate different candidate inputs and submit only a favorable one.



Schema 2. Two manipulation points: attackers can test inputs, while validators can influence inclusion.

msg.sender AND prevrandao are knowable

```
function getRandomNumber() external view returns (uint256) {
  uint256 randomNumber = uint256(
    keccak256(abi.encodePacked(msg.sender, block.prevrando, block.timestamp)));
  return randomNumber;
}
```

The fix: use Chainlink VRF or another verifiable randomness oracle. The randomness is generated after the request and revealed later with a cryptographic proof, preventing callers/validators from pre-selecting a favorable outcome.

RECOMMENDED: verifiable randomness with Chainlink VRF

```
// Request randomness; the result is provided later.
function requestRandomness() external returns (uint256 requestId) {
    return s_vrfCoordinator.requestRandomWords(
        VRFV2PlusClient.RandomWordsRequest({
            keyHash: KEY_HASH, subId: SUBSCRIPTION_ID,
            requestConfirmations: 3,
            callbackGasLimit: 100000, numWords: 1,
            extraArgs: VRFV2PlusClient._argsToBytes(
                VRFV2PlusClient.ExtraArgsV1({nativePayment:false})
            )
        })
    );
}

// Called later by the VRF Coordinator with verified randomness.
function fulfillRandomWords(uint256 requestId, uint256[] calldata randomWords) internal override
{
    uint256 randomNumber = randomWords[0];
    // Use randomNumber here.
}
```

1.11 code-quality habits that prevent bugs before they exist

None of these are exploits on their own — they're the difference between code an auditor can review quickly and code where real bugs hide in the noise.

“latex

- **Pin your pragma; don't float it.** `pragma solidity ^0.8.20;` allows the contract to compile with any compatible 0.8.x compiler version, including versions released after the code was tested or audited. Use an exact version when reproducibility matters: `pragma solidity 0.8.20;`
- **Name magic numbers.** `if (fee > 100)` tells a reviewer nothing about *why* 100. `uint256 constant MAX_FEE_BPS = 100;` makes the meaning and intent explicit, reducing the chance of incorrect assumptions or future mistakes.
- **Use consistent variable-name prefixes.** A common, audit-friendly convention is `s_` for storage variables, `i_` for immutables, and a leading underscore for function parameters that intentionally share a name with a storage variable.
- **Pin exact dependency versions.** A floating dependency range can silently introduce a compromised, vulnerable, or breaking update between the last audit and deployment. Pinning the exact dependency revision makes the build reproducible and ensures the audited code is the code being deployed.
- **Mark unchanging variables** `constant` **or** `immutable`. If a value is known at compile time and never changes, use `constant`; if it is assigned once during contract creation and never changes afterward, use `immutable`.
- **Arithmetic & Unsafe Casts and Rounding Errors**, it is valuable to examine their underlying mechanics:
 - **Overflow/Underflow:** Boundary failures where values exceed or fall below representable limits. Solidity ≥ 0.8 reverts by default, excluding `unchecked` blocks and explicit narrowing casts.
 - **Precision Loss:** Occurs independently of boundaries via integer division truncation (e.g., $5 / 2 = 2$). Repeated truncation can accumulate into significant economic discrepancies if systematically exploitable.

02

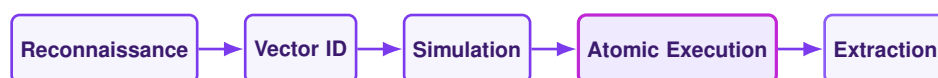
The Threat Landscape

An empirical analysis of historical exploit vectors, capital drain mechanics, and architectural vulnerabilities.

2 The Threat Landscape

Catastrophic protocol failures consistently stem from a predictable taxonomy of systemic flaws. Deconstructing the mechanics of an attack vector is foundational to engineering robust defenses.

2.1 Exploit Lifecycle and Execution Mechanics



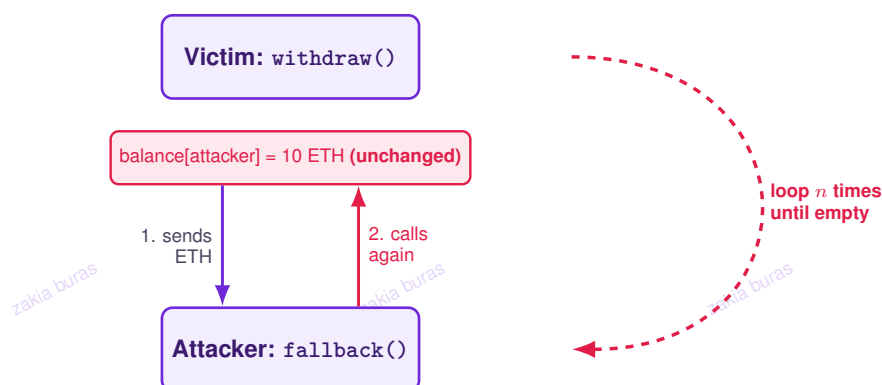
Schema 3. The multi-stage lifecycle of an advanced on-chain exploit.

Because blockchain state is public, attackers can copy it onto their own machine and rehearse the exact attack there first — checking it actually drains funds — before ever sending the real transaction. What you see on-chain is never a first attempt; it's a rehearsed, guaranteed win.

2.2 Lessons written in history — The DAO (2016)

The DAO was a large investment fund controlled by a smart contract. Its withdrawal function **sent ETH before updating the user's balance**. An attacker exploited this by recursively calling `withdraw`, draining roughly a third of the fund before the balance was updated. Ethereum later underwent a *hard fork* to reverse the theft, while the original chain became **Ethereum Classic**.

Root cause: *external call before state update, enabling reentrancy and resulting in massive fund loss.*



Schema 4. Reentrancy: balance never updates between calls, so the loop repeats until the contract is drained.

2.2.1 How to prevent it: the developer's job

Re-entrancy vectors share a common root: relinquishing execution control to an external contract before internal accounting and state transitions are fully finalized.

- **Checks-Effects-Interactions (CEI) Pattern:** Always update internal state variables and balances *before* executing any external calls. Pair this architectural ordering with a re-entrancy guard (e.g., OpenZeppelin's `nonReentrant`) as a secondary defense layer.
- **Transitioning from Push to Pull Payments:** Avoid iterating through arrays to distribute funds directly. A single failing recipient or malicious fallback can block the entire payout batch; instead, implement withdrawal patterns where users claim their own allocations.
- **Handling Advanced Token Callbacks:** Standards like ERC-777, ERC-1155, or custom hooks invoke receiver logic mid-transfer. Apply strict state-first update discipline to token-transfer workflows, not just native ETH transfers.
- **Mitigating Read-Only Re-entrancy:** External protocols reading state via `view` functions during an intermediate transaction step can ingest corrupted data. Finalize all state modifications completely prior to external interactions.

2.2.2 Code example: applying the fix

Standard Case — CEI Pattern and Re-entrancy Guard:

State First, Send Last, Lock the Function

```
function withdraw() public nonReentrant {
    uint256 bal = balances[msg.sender];
    balances[msg.sender] = 0;           // 1) Update state FIRST
    totalPool -= bal;                   // (Mitigates read-only re-entrancy)

    (bool success, ) = msg.sender.call{value: bal}(""); // 2) External call LAST
    require(success, "Transfer failed");
}

// Note: For token transfers with callbacks (ERC-777/1155), always clear internal
// balances before executing transfer routines to prevent hook-based re-entrancy.
```

Batch Distribution — Transitioning from Push to Pull Payments:

Pull Over Push Pattern

```
mapping(address => uint256) public owed;

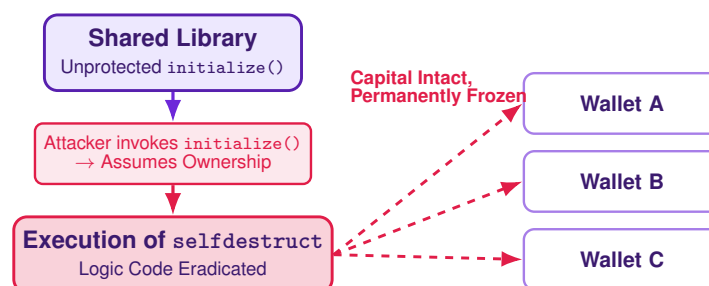
function claim() public nonReentrant {
    uint256 amount = owed[msg.sender];
    require(amount > 0, "No funds owed");

    owed[msg.sender] = 0;               // Clear obligation before transfer

    (bool success, ) = msg.sender.call{value: amount}("");
    require(success, "Transfer failed"); // Failures impact only the individual caller
}
```

2.3 Lessons Written in History: The Parity Multisig Freeze (2017)

Hundreds of Parity multisig wallets shared a single **logic library contract** via `delegatecall` to save gas. An attacker exploited an unprotected `initialize()` function to assume ownership of the standalone library contract and executed `selfdestruct`. This permanently erased the shared code. While funds remained trapped inside the proxy wallets, no logic remained to route transactions, freezing hundreds of thousands of ETH indefinitely.



Schema 5. Parity: Hijacking a shared implementation contract permanently bricked all dependent proxies.

2.3.1 How to prevent it: the developer's job

While individual defenses (such as initializers and the deprecation of `selfdestruct`) were established previously, upgradeable logic contracts demand proactive hardening at birth:

- **Locking Implementation Contracts:** Always execute `_disableInitializers()` directly within the logic contract's native constructor. This permanently locks the implementation contract at deployment, preventing anyone from initializing or hijacking it independently of its proxies.
- **Post-Deployment Verification:** Programmatically verify initialization states and ownership bindings immediately following deployment cycles.

Implementation-Level Initialization Lock

```
contract WalletLogic is Initializable {
    address public owner;

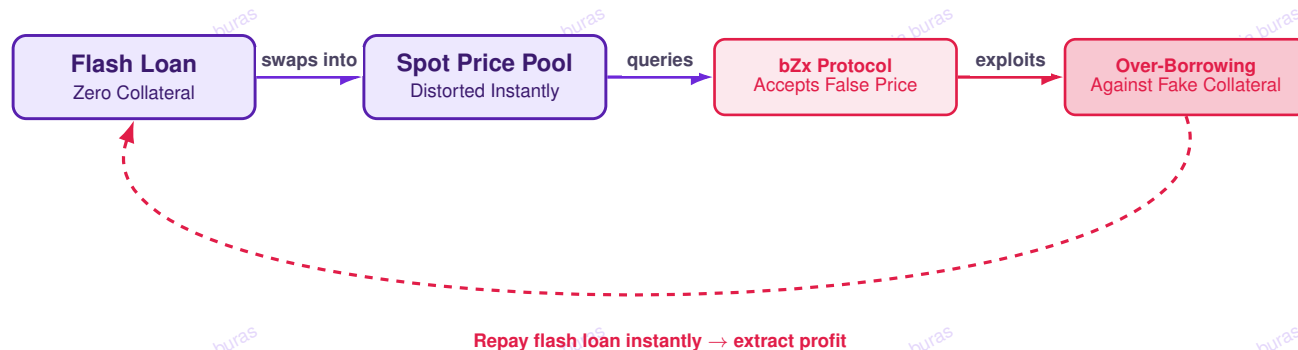
    function initialize(address _owner) public initializer {
        owner = _owner; // Executed securely through the proxy
    }

    constructor() {
        _disableInitializers(); // Locks the implementation contract at birth
    }
}
```

2.4 Lessons Written in History: The bZx Protocol Exploit (2020)

A **flash loan** permits borrowing arbitrary capital with zero collateral under a strict operational constraint: the loan must be fully repaid within the exact same atomic transaction. Because Ethereum transactions are atomic (succeeding completely or reverting entirely), lenders face zero principal risk, eliminating the need for traditional collateral.

In 2020, the bZx lending protocol priced asset collateral dynamically using spot reserves from a single on-chain liquidity pool. An attacker leveraged a flash loan to violently distort that pool's spot price, borrowed against the artificially inflated collateral value, repaid the flash loan, and pocketed roughly \$350,000 in a single atomic transaction.



Schema 6. bZx Exploit: A single atomic transaction manipulates the oracle, drains the pool, and exits before block closure.

2.4.1 How to prevent it: the developer's job

- **Eliminate Single-Pool Spot Pricing:** Never rely on instantaneous spot prices from isolated liquidity pools. An attacker with flash-borrowed capital can trivially shift spot prices within a single block.
- **Deploy Manipulation-Resistant Oracles:** Integrate Time-Weighted Average Prices (TWAP) or aggregated, decentralized feeds (e.g., Chainlink) that cannot be swung by isolated, single-block trades.
- **Implement Bounds and Sanity Checks:** Reject price movements that exceed realistic statistical thresholds within a single block window.
- **Assume Flash Liquidity:** Never build security assumptions around the premise that an attacker lacks capital. Flash loans render capital constraints obsolete.

2.4.2 Code example: fixing the oracle

Using the current spot price can be dangerous because an attacker can manipulate it within a single transaction. A **TWAP** uses prices from a period of time instead of one instant, making short-term manipulation much harder.

#VULNERABLE

```
function getPrice() public view returns (uint256) {
```

```

    return pool.getSpotPrice(); // VULNERABLE: Instantaneous spot price, easily manipulated
}

```

Using a TWAP Oracle Instead of Spot Pricing

```

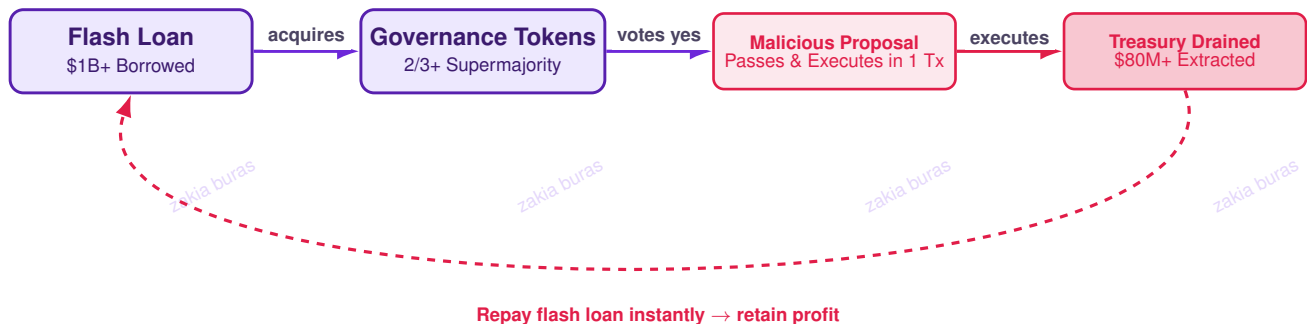
function getPrice() public view returns (uint256) {
    return oracle.consultTWAP(pool, 30 minutes);
    // SECURE: A single-block spike becomes a negligible data
    // point across a 30-minute window.
}

```

2.5 Lessons Written in History: The Beanstalk Farms Exploit (2022)

The Beanstalk protocol permitted emergency governance proposals to pass instantly if a token holder attained a two-thirds supermajority. Because voting weight was calculated using instantaneous token balances, an attacker could acquire massive voting power via flash loans within a single block.

The attacker flash-loaned over \$1 billion, secured supermajority voting control, passed a malicious proposal disguised as a charitable donation, and executed it instantly. The exploit drained the protocol's liquidity pools, netting the attacker approximately **\$80 million**.



Schema 7. Beanstalk Exploit: Flash-borrowed voting weight bypasses time-locks to pass and execute a malicious proposal atomically.

2.5.1 How to prevent it: the developer's job

- **Enforce Historical Snapshots:** Never calculate voting power using live token balances. Anchor voting weight to historical token checkpoints from a preceding block (e.g., OpenZeppelin's ERC20Votes).
- **Mandate Timelocks:** Prevent instantaneous execution. Enforce strict operational delays between proposal submission, voting closure, and final execution.
- **Secure Emergency Paths:** Eliminate fast-track governance bypasses that permit raw token thresholds to override review periods without multi-signature or decentralized timelock safeguards.

2.5.2 Code example: snapshot voting instead of live balance

Relying on instantaneous balances exposes governance mechanisms to flash-loan manipulation. Enforcing historical checkpoints renders single-block capital injections ineffective.

#VULNERABLE

```

function votingPower(address user) public view returns (uint256) {
    return govToken.balanceOf(user); // Live balance is vulnerable to flash-loan inflation
}

```

SECURE: Historical checkpoint guarantees flash-loan immunity.

```

function votingPower(address user, uint256 proposalBlock) public view returns (uint256) {
    return govToken.getPastVotes(user, proposalBlock);
}

```

2.6 Lessons written in history — Euler Finance (2023)

Euler Finance was a non-custodial lending protocol where users deposited assets for eTokens (collateral) and borrowed against them, receiving dTokens (debt).

On March 13, 2023, an attacker used a flash loan to open a leveraged position, then called `donateToReserves()` to donate their eTokens to the protocol reserves. Unlike every other balance-changing function, this function skipped the protocol's core solvency check. The donation pushed the attacker's account into insolvency without reverting, allowing them to *self-liquidate* their underwater position and pocket the liquidation discount. Repeated across multiple markets, this drained roughly **\$197 million** in minutes.

Root cause: A newly added post-audit function bypassed the global solvency check, breaking the protocol's core invariant ("no account shall remain insolvent").

2.6.1 How to prevent it: the developer's job

- **Enforce invariants globally.** A core safety invariant is only as strong as its weakest enforcement point; missing a check in a single function invalidates the entire model.
- **Re-audit post-deployment changes.** Functions added during later upgrades require the same rigorous scrutiny and invariant checks as the original codebase.
- **Block self-liquidation paths.** Restrict logic so users cannot profit from liquidating their own accounts.
- **Maintain incident response readiness.** Euler's successful on-chain negotiation and recovery of 90% of funds highlight the value of clear communication channels during a crisis.

2.7 Lessons written in history — Cetus Protocol (2025)

Cetus was a major concentrated-liquidity DEX on the Sui blockchain. Adding liquidity required a complex math calculation guarded by a custom overflow check called `checked_shlw`.

On May 22, 2025, an attacker requested massive liquidity within an extremely narrow price range. Due to a flaw, the check failed to catch the overflow, silently wrapping the required deposit down to just **1 token** while crediting the full, massive position. The attacker deposited that 1 token and drained real reserves against the phantom balance, stealing roughly **\$223–260 million** in under 15 minutes.

Even though Cetus passed three audits by MoveBit, OtterSec, and Zellic, the bug went unnoticed because it was tucked away in a shared third-party math library that fell outside the core review scope.

Root cause: A flawed custom overflow check in shared library code that silently truncated values instead of reverting, escaping detection across multiple audits.

2.7.1 How to prevent it: the developer's job

- **Audit third-party math libraries.** Never assume shared code is safe just because it is popular; verify every line that handles value or risk.
- **Test extreme boundaries.** Use fuzz testing to catch unusual edge cases that manual unit tests miss.
- **Don't trust hand-rolled safety checks blindly.** Custom helper functions can easily contain logic errors.
- **Explicitly scope shared code in audits.** Ensure external libraries are actively reviewed rather than assumed to be someone else's responsibility.

2.8 Lessons written in history — Kelp DAO (2026)

Kelp DAO issues rsETH, a liquid restaking token backed by ETH restaked through EigenLayer. To move that value across chains, Kelp relied on LayerZero's cross-chain messaging infrastructure.

In April 2026, attackers exploited a misconfigured cross-chain verification setup in that LayerZero integration, minting or redirecting funds without a legitimate corresponding deposit. About **18% of the entire rsETH supply** was affected, roughly **\$292 million**. Because rsETH was used as collateral across major lending markets, the

exploit didn't stay contained—it triggered a liquidity crunch and a wave of withdrawals across other protocols that had nothing to do with the original bug.

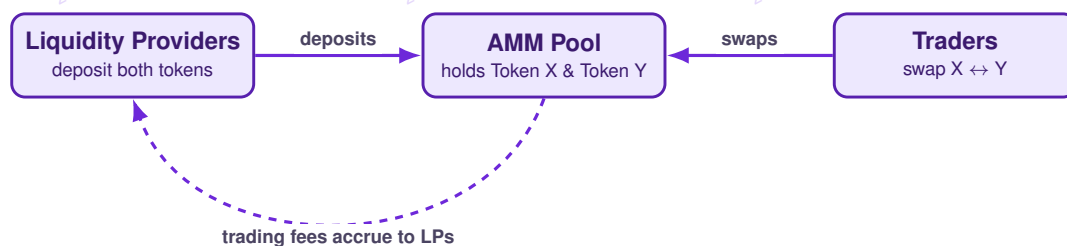
Root cause: A misconfigured trust boundary in cross-chain message verification (specifically a vulnerable 1-of-1 DVN setup targeted via infrastructure RPC poisoning), compounded by how deeply the affected token was integrated elsewhere.

2.8.1 How to prevent it: the developer's job

- **Treat every cross-chain integration as a new trust boundary**, not a plumbing detail.
- **Understand exactly what your messaging layer verifies, and what it assumes.** “Modular” security frameworks let you configure verification loosely; relying on a single verifier (like a 1-of-1 DVN setup) creates an immediate single point of failure.
- **Model contagion, not just direct loss.** A token used as collateral everywhere means a bug in one place becomes a liquidity crisis everywhere it's integrated—size your exposure caps accordingly.
- **Don't assume someone else's audit covers your integration.** The underlying restaking protocol and the messaging layer may each be individually audited while the specific way they're wired together isn't.

2.9 AMMs and the Constant Product Invariant

Automated Market Makers (AMMs) replace traditional order books with liquidity pools governed by mathematical formulas. Liquidity Providers (LPs) supply token pairs, while traders swap directly against the pool, paying fees that accrue to LPs.



Schema 8. The AMM core loop: LPs supply liquidity, traders pay fees, and invariant rules regulate state.

The Core Mathematical Invariant: Uniswap-v2 style pools enforce that the product of token reserves remains constant (or increases due to fees) across every swap:

$$x \cdot y = k$$

where x and y represent the pool's token reserves, and k is the invariant product. When a trader deposits Δx to receive Δy , the post-trade balances must satisfy:

$$(x + \Delta x \cdot (1 - \rho))(y - \Delta y) = k$$

where ρ represents the fee percentage. Solving for the output amount Δy yields:

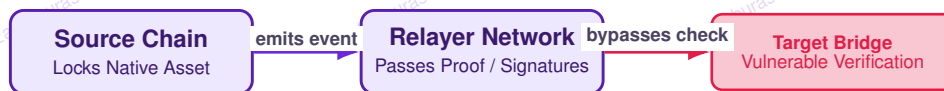
$$\Delta y = \frac{y \cdot \Delta x \cdot (1 - \rho)}{x + \Delta x \cdot (1 - \rho)}$$

Security Relevance: For an Automated Market Maker, an invariant equation like $x \cdot y = k$ represents its entire economic lifeblood: the math that guarantees the pool remains solvent. Fuzz and invariant test suites (§6.4) test this by throwing thousands of random trades, swaps, and flash loans at the AMM pool to check if k ever breaks. If an overflow, precision loss, or rounding error silently violates this rule like the bug that let Cetus extract millions from a liquidity pool using just one token, the AMM's internal accounting becomes completely disconnected from reality, letting attackers drain its reserves.

2.10 Cross-Chain Bridge Vulnerabilities

Cross-chain bridges move assets between different blockchains. Typically, the bridge **locks assets on one chain** and releases or mints equivalent assets on another. Because bridges often hold large amounts of funds, a flaw in their verification system can allow attackers to create or withdraw assets without proper backing.

- **Validator Set Compromise:** Many bridges rely on a small group of signers to approve transfers. If an attacker compromises enough private keys, they can approve fake transfers and steal the bridge's locked funds.
- **Light-Client Verification Bugs:** Some bridges verify another blockchain's data directly on-chain. A bug in this verification can make the bridge accept fake block headers or fraudulent state as valid.
- **Optimistic Challenge Window Flaws:** Optimistic bridges assume submitted data is valid unless someone challenges it within a specific time. If the challenge period is too short, the required bond is too small, or nobody detects the fraud, a malicious update may become final.



Schema 9. Bridge Architecture: A compromised validator quorum or flawed verification allows malicious state injection.

2.10.1 Code Example: Vulnerable vs. Secure Cross-Chain Verification

Vulnerable :Missing nonce check, replayable messages, and unvalidated signatures

```
function executeCrossChainTransfer(bytes calldata payload, bytes calldata signature) external {
    // Flaw 1: Missing signature threshold verification (accepts a single signature)
    require(ECDSA.recover(keccak256(payload), signature) == trustedRelayer, "Invalid signature");

    // Flaw 2: No replay protection (the same payload can be executed multiple times)
    // Flaw 3: No nonce check to enforce ordering

    _processPayload(payload);
}
```

Secure :Enforce threshold signatures, re-entrancy locks, and nonce replay protection

```
mapping(bytes32 => bool) public processedMessages;

function executeCrossChainTransfer(bytes calldata payload, bytes[] calldata signatures, uint256 nonce) external nonReentrant {
    // Create a unique fingerprint for this specific transfer instruction
    bytes32 messageHash = keccak256(abi.encodePacked(payload, nonce, block.chainid));

    // Replay protection: reject messages that have already been executed
    require(!processedMessages[messageHash], "Message already processed");
    processedMessages[messageHash] = true;

    // Verify that a valid mathematical quorum of authorized validators signed the payload
    require(ValidatorLib.verifyThreshold(messageHash, signatures), "Invalid validator quorum");

    // Safely decode and execute the verified cross-chain payload
    _processPayload(payload);
}
```

2.10.2 How to prevent it: the developer's job

- **Trustless Verification:** Prioritize cryptographic light clients (such as Zero-Knowledge consensus proofs) over federated multi-signature committees.

- **Replay and Nonce Protection:** Enforce strict nonces and cryptographic message fingerprinting to ensure transfer instructions cannot be replayed across chains.
- **Robust Economic Guarantees:** Ensure optimistic challenge windows provide adequate time and sufficient watcher economic incentives to successfully dispute invalid state roots.

2.11 Liquid Staking and Restaking Risk

Restaking protocols (such as **EigenLayer**) let you reuse your staked tokens to secure multiple external services at once. While this creates extra yield, it introduces a dangerous domino effect: a single hack or penalty can wipe out funds across every service your tokens are backing.

2.11.1 Understanding Restaking and Slashing Mechanics

Normally, your staked ETH secures only the base Ethereum blockchain. Restaking acts like an extension cord, plugging your staked capital into external modules (such as oracles or bridges) so they inherit security without building it from scratch. However, this introduces severe systemic vulnerabilities:

- **The Opt-in Trap (Shared Security Models):** Node operators choose which external services to support. If an operator opts into a poorly coded service that gets exploited, the underlying staked capital can be slashed or burned for a fault your chosen strategy never committed.
- **Slashing Cascade Mechanics:** Because the same capital backs multiple modules simultaneously, a widespread bug or protocol failure can trigger penalties across several networks concurrently, exponentially compounding losses.
- **Rehypothecation Loops (The Leverage Tower):** Staked and receipt tokens are frequently deposited into lending markets to borrow leverage, purchase more assets, and restake recursively. A sudden price shock or depeg can trigger cascading liquidations across every leveraged layer.

2.11.2 How to prevent it: the developer's job

- **Additive Risk Modeling:** Treat every new service you opt into as an extra layer of risk, not just free money.
- **Deterministic Slashing Audits:** Ensure slashing rules are transparent and cannot be arbitrarily changed by project governance later.
- **Leverage and Exposure Caps:** Set strict deposit limits on liquid restaked assets to prevent cascading failures during market stress.

2.11.3 Code Example: Validating Restaked Collateral and Slashing States

SECURE: Block deposits from flagged operators and enforce strict risk limits

```
mapping(address => uint256) public restakedBalances;
mapping(address => bool) public slashedOperators;
uint256 public constant MAX_EXPOSURE_LIMIT = 1000 ether;
function depositRestakedCollateral(address operator, uint256 amount) external nonReentrant {
    // Step 1: Reject operators who have already been penalized or flagged for bad behavior
    require(!slashedOperators[operator], "Operator has active slashing flags");
    // Step 2: Prevent a single operator from hoarding too much risk in our protocol
    require(restakedBalances[operator] + amount <= MAX_EXPOSURE_LIMIT, "Exceeds protocol
    exposure cap");
    restakedBalances[operator] += amount;
    // Step 3: Safely process the restaked deposit mapping
    _processRestakeDeposit(operator, amount);
}
```

2.12 Layer 2 & Rollup Security Architecture

Layer 2 networks scale execution by moving transactions off-chain, but they shift the trust model from decentralized Layer 1 consensus to complex cryptographic and sequencer assumptions.

- **Sequencer Centralization & Censorship:** Most L2s rely on a single centralized sequencer to order transactions. If compromised, attackers can front-end trades or censor user withdrawals. Protocols require L1-to-L2 escape hatches (forced transaction submission via L1) to guarantee users can exit independently of sequencer cooperation.
- **Fraud Proofs vs. Validity Proofs:** Optimistic rollups rely on a challenge window (often 7 days) during which anyone can submit fraud proofs; a bug in the dispute resolution contract risks full state corruption. ZK-rollups eliminate the delay using cryptographic validity proofs, but place total trust in circuit correctness and trusted setup ceremonies.
- **Cross-Domain Replay Protection:** Messaging bridges between L1 and L2 must enforce strict domain separators and nonce tracking to prevent state updates or withdrawal messages from being replayed across different networks.

2.12.1 How to prevent it: the developer's job

- **Decentralized Sequencer Networks:** Transition from single-sequencer architectures to shared or decentralized sequencer committees (such as threshold-encrypted or rotating leader protocols) to eliminate single points of censorship and front-running.
- **Rigorous Circuit and Proof Auditing:** For ZK-rollups, subject all zero-knowledge circuits, verifier smart contracts, and trusted setup ceremonies to exhaustive cryptographic audits and formal verification to prevent soundness bugs.
- **Guaranteed L1 Escape Hatches:** Implement robust, trustless forced-inclusion mechanisms allowing users to submit withdrawal or transaction requests directly through Layer 1 smart contracts if the sequencer halts or attempts censorship.

2.13 Account Abstraction (ERC-4337) and Passkey Validation Security

ERC-4337 replaces traditional EOAs with smart contract wallets via a two-step process: **Simulation** (bundlers verify validity and gas) followed by **Execution** on-chain.

2.13.1 The core problem: changing data can break validation

If `validateUserOp` reads volatile data like external prices or balances that change between steps, transactions can pass **simulation** and fail on-chain, allowing attackers to cause uncompensated gas waste known as validation griefing.

VULNERABLE: The price may change after simulation.

```
function validateUserOp(UserOperation calldata op, bytes32 hash) external returns (uint256) {
    require(externalPriceFeed.getPrice() > threshold, "Price gate failed");
    return _verifySignature(op, hash) ? 0 : 1;
}
```

2.13.2 How to prevent it: keep validation deterministic

Keep `validateUserOp` deterministic by relying solely on static data like signatures and nonces, and inherit battle-tested libraries for complex features like passkeys.

SECURE: Validation depends only on the signature and hash

```
function validateUserOp(UserOperation calldata op, bytes32 hash) external returns (uint256) {
    return _verifySignature(op, hash) ? 0 : 1;
}
```


2.13.3 Malicious paymasters and bundler censorship

ERC-4337 also introduces two important trust assumptions:

- **Malicious paymasters.** Paymasters pay gas for users, so a vulnerable or malicious paymaster can cause failed transactions or lose its deposit. Choose trusted, well-audited paymasters.
- **Bundler censorship.** Bundlers decide which UserOps to include. A bundler can ignore a UserOp, so users should not depend on a single bundler. Multiple independent bundlers improve resilience against censorship.

These risks cannot be fully eliminated by the account contract itself. They must be addressed through **trusted infrastructure, multiple providers, and careful dependency selection**.

2.14 Front-Running and Sandwich Attacks

Sandwich attacks act as an automated tax on user trades by exploiting the **public mempool**. When a user submits a swap, a bot detects the pending order and executes a three-step attack:

1. **Front-Run (The Injection):** The bot monitors the public mempool, detects your pending swap, and pays a higher gas fee to have its own buy order executed right before yours, driving up the token's price.
2. **Victim Execution (The Trap):** Your transaction executes immediately after the bot's, forcing you to buy the token at the artificially inflated, expensive price.
3. **Back-Run (The Extraction):** In that exact same block, the bot sells the tokens it just bought at your higher price, pocketing the difference as guaranteed profit.



Schema 10. Sandwich Attack Flow: From mempool detection to multi-step transactional extraction.

2.14.1 How to prevent it: the developer's job

- **Enforce Strict Slippage Floors:** Never accept unconstrained token returns (`minOut = 0`). Require explicit minimum output thresholds to guarantee transactions revert if price manipulation exceeds tolerable bounds.
- **Implement Tight Deadlines:** Expire stale transactions promptly to prevent them from lingering in mempools where bots can calculate optimal sandwich parameters.
- **Utilize Private RPCs and Relays:** Route transactions through private channels (such as Flashbots Protect) or zero-knowledge mempools to keep trades invisible to searcher bots until successfully included on-chain.

2.14.2 Code example: closing the window

Unconstrained slippage invites extraction bots; programmatic price floors ensure manipulated trades revert instantly.

#VULNERABLE

```
// VULNERABLE: minOut = 0 accepts any execution price, inviting sandwich bots
router.swapExactTokensForTokens(amountIn, 0, path, msg.sender, block.timestamp);
```

Enforcing a Strict Slippage Floor

```
// SECURE: Reverts instantly if price deviation exceeds allowed basis points
uint256 minOut = expectedOut * (10000 - maxSlippageBps) / 10000;
router.swapExactTokensForTokens(amountIn, minOut, path, msg.sender, block.timestamp);
```

2.15 Defensive Engineering Takeaways

All the major disasters came from ordinary architectural and implementation oversights, not exotic cryptography. Turn these principles into non-negotiable habits for every smart contract you write:

- **Threat-Model First:** Before writing a single line of logic, enumerate every function, identify who can call it, and map out what inputs or capital they control.
- **Assume a Hostile Caller:** Your source code is public, your bytecode is readable, and flash loans hand any stranger unlimited capital for a single block. Design as if an active attacker is guaranteed to invoke every function.
- **Follow Checks-Effects-Interactions (CEI):** Always update internal state and balances *before* making any external call or token transfer, and wrap state-changing entry points in reentrancy guards — *The DAO (2016)*.
- **Lock Libraries and Implementation Contracts:** Call `_disableInitializers()` in constructors for implementation contracts and shared libraries, and eliminate unconstrained `selfdestruct` paths — *Parity Multisig (2017)*.
- **Never Trust a Single Price Source:** Spot prices from isolated liquidity pools can be distorted instantly via flash loans; mandate Time-Weighted Average Prices (TWAP) or aggregated oracles — *bZx (2020)*.
- **Anchor Governance to Historical Snapshots:** Never calculate voting power using live token balances that can be inflated via flash loans within a single block — *Beanstalk (2022)*.
- **Enforce Strict Slippage and Nonce Protection:** Never accept unconstrained token returns ($minOut = 0$); mandate strict price floors, expiry deadlines, and unique message nonces to mitigate sandwich attacks and cross-chain replays.
- **Mitigate Bridge and Restaking Exposure:** Treat cross-chain verifications as consensus-critical, enforce trust-minimized light clients, and size liquid restaking exposure caps to prevent cascading liquidation loops.
- **Design for Failure:** You will ship a bug eventually. Global pause switches, withdrawal rate-limits, and invariant checks dictate whether a bug results in a minor incident or a total treasury loss.
- **Never Roll Your Own Primitives:** Access control, math, token standards, and guards are solved problems; inherit battle-tested libraries (such as OpenZeppelin) instead of rewriting them.
- **Test Like an Adversary:** Fork mainnet and attempt to break your own system using fuzzing, invariant tests, transaction reordering, and flash-loan simulations rather than just verifying the happy path.
- **Gate Mainnet Deployments Behind Rigorous Audit:** Require formal verification, independent peer security reviews, and active bug bounties before exposing real user capital to production environments.

03

The Vulnerability Catalogue

A field reference: the recurring bug classes, each with a broken example, its fix, and a one-line mitigation.

3 The Vulnerability Catalogue

Sections 1–2 taught the core mechanics in depth. This section is your pre-deployment quick-reference for identifying and patching recurring flaw classes, ordered from critical architectural exploits down to medium-severity execution risks.

1 · Reentrancy

SWC-107

CRITICAL

An external call made before the state update lets the callee re-enter and drain the balance. **See also:** §2 (The DAO), with four worked fixes.

#VULNERABLE

```
// VULNERABLE: sends ETH before clearing the balance
function withdraw() public {
    uint256 bal = balances[msg.sender];
    (bool ok, ) = msg.sender.call{value: bal}("");
    require(ok);
    balances[msg.sender] = 0; // too late
}
```

state before call, plus a reentrancy guard

```
function withdraw() public nonReentrant {
    uint256 bal = balances[msg.sender];
    balances[msg.sender] = 0; // state FIRST
    (bool ok, ) = msg.sender.call{value: bal}("");
    require(ok);
}
```

Mitigation: Update state before external calls; add `nonReentrant`; prefer pull payments.

2 · tx.origin Authentication

SWC-115

CRITICAL

Authorizing with `tx.origin` trusts the original user, so a malicious middle contract can phish the owner. **See also:** §1 (Message calls).

#VULNERABLE

```
require(tx.origin == owner); // a phishing contract calling as owner: PASSES
```

authenticate the direct caller

```
require(msg.sender == owner); // the malicious contract is not the owner:
REVERTS
```

Mitigation: Never authorize on `tx.origin`; check `msg.sender`.

3 · Delegatecall & Storage Collision

SWC-112

CRITICAL

`delegatecall` runs another contract's code in your storage; a hostile or mismatched target rewrites your variables. **See also:** §1 (Storage & `delegatecall`).

#VULNERABLE

```
Logic Contract: slot 0 = address owner;
Proxy Contract: slot 0 = uint256 total;
// delegatecall writes to "owner" -- overwrites "total" instead
```

inherit an audited proxy with a matched, versioned layout

```
// Use OpenZeppelin's TransparentUpgradeableProxy / UUPS,  
// which enforce identical, EIP-1967-routed storage layouts.
```

Mitigation: Never delegatecall untrusted addresses; align storage layouts; inherit an audited proxy.

4 · Uninitialized Proxy

SWC-118

CRITICAL

An unprotected `initialize()` lets anyone claim ownership of an upgradeable contract. **See also:** §2 (Parity multisig freeze).

#VULNERABLE

```
// VULNERABLE: no initializer modifier, callable by anyone, repeatedly  
function initialize(address _owner) public {  
    owner = _owner;  
}
```

Initializable + a disabled constructor path

```
function initialize(address _owner) public initializer {  
    owner = _owner;  
}  
  
constructor() { _disableInitializers(); }
```

Mitigation: Use `Initializable`; call `_disableInitializers()` on the implementation; verify ownership after deploy.

5 · Cross-Chain Bridge Verification Failure

Bridge

CRITICAL

Accepting fake transfer proofs or compromised multisig signatures lets attackers mint tokens out of thin air. **See also:** §2 (Cross-Chain Bridges).

#VULNERABLE

```
// VULNERABLE: no nonce, no threshold check  
function release(address to, uint256 amt) external {  
    token.transfer(to, amt);  
}
```

nonce + threshold signature verification

```
require(!processedMessages[hash], "replayed");  
processedMessages[hash] = true;  
require(ValidatorLib.verifyThreshold(hash, sigs), "bad quorum");
```

Mitigation: Enforce nonces, message fingerprinting, and threshold signatures; favor ZK light clients.

6 · Broken Access Control

SWC-105

CRITICAL

A privileged function lacks an authorization check, allowing anyone to seize ownership or execute admin actions.

#VULNERABLE

```
// VULNERABLE: Anyone can call and overwrite the owner  
function setOwner(address newOwner) public { owner = newOwner; }
```

OpenZeppelin `AccessControl` / `TimelockController` / `Multi-Sig`

```
function criticalAdminAction() external onlyRole(ADMIN_ROLE) {  
    // Protected by granular RBAC role checks  
}
```

Mitigation: Choose the right OpenZeppelin model: `Ownable2Step` for single-owner contracts, `AccessControl` for multi-role systems. For high-value protocols, require a `TimelockController` and assign control to a `Multi-Sig`.

7 · Oracle Manipulation

DeFi **CRITICAL**

A contract relies on a single-block spot price that can be artificially skewed within an atomic transaction using flash loans.

#VULNERABLE

```
(uint r0, uint r1,) = pair.getReserves();  
return r1 * 1e18 / r0;
```

Chainlink with freshness checks, or a Uniswap TWAP

```
(, int price,, uint updatedAt,) = feed.latestRoundData();  
require(price > 0 && block.timestamp - updatedAt < 3600, "Stale price");
```

Mitigation: Never price assets off a single on-chain spot reserve. Use aggregated decentralized feeds (e.g., Chainlink) with heartbeat/staleness validation, or implement TWAPs across a multi-block window.

8 · Unprotected `selfdestruct`

SWC-106 **CRITICAL**

A `selfdestruct` function left without an access check lets any caller destroy the contract and force-send its balance — exactly the flaw that let Parity's shared library get bricked. **See also:** §1 (`selfdestruct` risks); §2 (Parity multisig freeze).

#VULNERABLE

```
function kill() public { selfdestruct(payable(msg.sender)); }
```

avoid `selfdestruct`; use a pausable pattern

```
function pause() external onlyRole(GUARDIAN_ROLE) { _pause(); }
```

Mitigation: Avoid `selfdestruct` entirely; if a kill switch is required, gate it behind a multisig/timelock and prefer a pausable pattern.

9 · Malicious Insider

Governance **CRITICAL**

A team member or privileged party with *legitimate* access uses it maliciously — this is a behavior risk, not a code bug, so it isn't caught by an audit of the code itself. **Mitigation:** Require multisig for any privileged action, not a single trusted signer; add a timelock so even a legitimate-looking malicious action is visible before it executes; apply least privilege so no one person's access alone can drain funds.

10 · Supply Chain Attack

Infra **CRITICAL**

A compromised dependency — an npm package, an imported Solidity library, a compiler plugin — injects malicious code into your project without a bug in any code you wrote yourself. **Mitigation:** Pin exact dependency versions and audit diffs on every upgrade; prefer well-known, widely-audited libraries (OpenZeppelin) over obscure

packages; verify package checksums/provenance in CI.

11 · Vault Inflation (first-depositor)

DeFi

HIGH

In an ERC-4626-style vault, the first depositor can donate assets to skew the share price and steal later deposits.

#VULNERABLE

```
shares = assets * totalSupply / totalAssets; // 0 supply -> attacker sets the
rate
```

FIXED

```
uint shares = _convertToShares(assets, Math.Rounding.Down);
require(shares > 0, "zero shares");
```

Mitigation: Use OpenZeppelin's ERC4626 with virtual offset; seed initial liquidity; reject zero-share mints.

12 · Restaking Slashing Cascade

DeFi / Restaking

HIGH

Restaking protocols allow a single staked principal to back multiple AVSs concurrently, causing correlated failures or bugs to trigger catastrophic cascading liquidations.

#VULNERABLE

```
function deposit(address operator, uint256 amount) external {
    balances[operator] += amount;
}
```

enforce strict exposure caps and operator health checks

```
require(!slashedOperators[operator], "Operator slashed");
require(balances[operator] + amount <= MAX_EXPOSURE_LIMIT, "Cap exceeded");
balances[operator] += amount;
```

Mitigation: Treat AVS opt-ins as compounding risk additions; enforce strict operator health flags and hard protocol exposure caps.

13 · Arithmetic & Unsafe Casts

SWC-101

HIGH

unchecked blocks and narrowing casts reintroduce overflow even in modern Solidity.

#VULNERABLE

```
unchecked { balances[msg.sender] -= amount; } // underflows to a huge number
```

FIXED

```
require(balances[msg.sender] >= amount, "insufficient");
balances[msg.sender] -= amount;
```

Mitigation: Compile with Solidity >= 0.8; avoid unchecked; use SafeCast for downcasts.

14 · Unchecked External Calls

SWC-104

HIGH

Low-level calls return a boolean instead of reverting; ignoring it hides silent failures.

#VULNERABLE

```
recipient.send(amount);           // return value ignored
token.transfer(to, amount);       // some tokens return false instead of reverting
```

FIXED

```
(bool ok,) = recipient.call{value: amount}(""); require(ok, "failed");
token.safeTransfer(to, amount);
```

Mitigation: Always check return values; wrap token transfers in SafeERC20.

15 · Signature Replay

SWC-121

HIGH

Signed messages without a nonce or chain ID can be replayed, or reused across chains.

#VULNERABLE

```
address signer = ecrecover(hash, v, r, s); // no nonce, no domain
require(signer == owner);
```

FIXED

```
bytes32 digest = _hashTypedDataV4(structHash);
require(!usedNonce[nonce], "replayed");
usedNonce[nonce] = true;
```

Mitigation: Use EIP-712 with a domain separator; track nonces; use OpenZeppelin ECDSA.

16 · Reward / Incentive Manipulation

DeFi

HIGH

An attacker games a protocol's reward or emissions formula directly — not the price feed — to extract far more rewards than their real economic contribution justifies.

#VULNERABLE

```
function claimReward(uint amount) external {
    reward[msg.sender] += amount * rewardRate;
}
```

base rewards on time-weighted or locked participation

```
require(block.timestamp >= depositTime[msg.sender] + MIN_LOCK, "too early");
reward[msg.sender] += stakedAmount[msg.sender] * rewardRate * duration;
```

Mitigation: Base rewards on time-weighted or minimum-lock participation, not instantaneous balances; simulate adversarial deposit/withdraw patterns before launch.

17 · Rounding Errors

SWC-101

HIGH

Distinct from overflow (#13): small precision losses in division or share calculations that compound, or that an attacker can trigger repeatedly to drain value a fraction at a time — see §2 (Cetus).

#VULNERABLE

```
uint share = (amount * totalShares) / totalAssets;
```


round in the protocol's favor, and check boundaries

```
uint share = Math.mulDiv(amount, totalShares, totalAssets, Math.Rounding.Down);
```

Mitigation: Always round in the protocol's favor, never the user's; use a battle-tested fixed-point math library; fuzz-test at extreme ratios.

18 · Improper Function Visibility

SWC-100

HIGH

A function is marked `public` or left with Solidity's implicit default when it should be `internal` or `private`, letting an attacker call it directly and bypass the checks a "front door" function would have enforced — see §1 (Function visibility).

#VULNERABLE

```
function _moveFunds(address to, uint amount) public {
    payable(to).transfer(amount);
}
```

default to the tightest visibility that works

```
function _moveFunds(address to, uint amount) internal {
    payable(to).transfer(amount);
}
```

Mitigation: Default to the tightest visibility that still works; never rely on a function being "hard to find" as security.

19 · Weak or Predictable Randomness

SWC-120

HIGH

Using on-chain values like `block.timestamp`, `blockhash`, or `block.prevrandao` (the post-Merge successor to the legacy `block.difficulty`, per EIP-4399) as a source of "randomness" — all of these are either predictable in advance or influenceable by the validator producing the block. **See also:** §1 (why `prevrandao` still isn't random); Slither's `weak-prng` detector flags this exact pattern automatically.

#VULNERABLE

```
function pickWinner() public view returns (uint) {
    return uint(keccak256(abi.encodePacked(block.timestamp, block.prevrandao))
) % 100;
}
```

use a verifiable off-chain randomness oracle

```
function requestRandom() external {
    requestId = COORDINATOR.requestRandomWords(keyHash, subId, ...);
}
```

Mitigation: Never derive randomness from block variables; use a verifiable randomness oracle (Chainlink VRF or equivalent).

20 · Unexpected Ether Balance

SWC-132

HIGH

A contract that does strict equality on `address(this).balance` can be permanently broken by anyone force-sending ETH: `selfdestruct` bypasses `receive()/fallback()` entirely and cannot be blocked. **See also:** §1 (`selfdestruct` risks — "Forced ETH Injection").

#VULNERABLE

```
function withdrawFees() external {
    require(address(this).balance == totalFees, "mismatch");
}
```

track owed funds internally; never assert exact contract balance

```
function withdrawFees() external {
    require(address(this).balance >= totalFees, "insufficient balance");
}
```

Mitigation: Never use == against a contract's own ETH balance; use >= or track internal accounting separately.

21 · Payable delegatecall Inside a Loop

Slither: delegatecall-loop

HIGH

A payable function that loops, calling delegatecall to other payable functions, lets msg.value be credited on every iteration.

#VULNERABLE

```
function multicalldata(bytes[] calldata calls) external payable {
    for (uint i = 0; i < calls.length; i++) {
        address(this).delegatecall(calls[i]);
    }
}
```

never combine payable functions with delegatecall loops

```
function multicalldata(bytes[] calldata calls) external {
    require(msg.value == 0, "no ETH in batched calls");
    for (uint i = 0; i < calls.length; i++) {
        address(this).delegatecall(calls[i]);
    }
}
```

Mitigation: Never mark a function payable if it loops and delegatecalls into other payable functions.

22 · Non-Standard (“Weird”) ERC20 Tokens

Token Integration

HIGH

Not every ERC20 behaves like the textbook standard — fee-on-transfer, rebasing, and ERC-777 reentrancy callbacks all silently break contracts that assume standard behavior.

#VULNERABLE

```
function deposit(uint256 amount) external {
    token.transferFrom(msg.sender, address(this), amount);
    balances[msg.sender] += amount; // wrong for fee-on-transfer tokens
}
```

measure the actual balance delta

```
function deposit(uint256 amount) external {
    uint256 before = token.balanceOf(address(this));
    token.transferFrom(msg.sender, address(this), amount);
    uint256 received = token.balanceOf(address(this)) - before;
    balances[msg.sender] += received;
}
```

Mitigation: Never assume a token behaves like a textbook ERC20; measure balance deltas; treat every new token

integration as untrusted (catalogue #10) until checked against a token integration checklist.

23 · Denial of Service (push loop)

SWC-113/128

MEDIUM

An unbounded loop, or one reverting recipient in a push loop, can freeze all payouts. **See also:** §2 (pull vs push).

#VULNERABLE

```
function payAll(address[] calldata recipients) external {
    for (uint i = 0; i < recipients.length; i++) {
        payable(recipients[i]).transfer(1 ether); // one revert blocks all
    }
}
```

pull-over-push: each recipient claims their own funds

```
function claim() external {
    uint256 amount = owed[msg.sender];
    owed[msg.sender] = 0;
    (bool ok, ) = msg.sender.call{value: amount}("");
    require(ok);
}
```

Mitigation: Bound every loop; favour pull-over-push; never let one participant block others.

24 · Front-Running & MEV

SWC-114

MEDIUM

The public mempool lets attackers reorder trades to sandwich or snipe them. **See also:** §2 (Sandwich attack).

#VULNERABLE

```
swap(tokenIn, tokenOut, amountIn); // no slippage bound, no deadline
```

FIXED

```
require(amountOut >= minAmountOut, "slippage");
require(block.timestamp <= deadline, "expired");
```

Mitigation: Enforce slippage limits and deadlines; use commit-reveal or private relays.

25 · ERC-20 Approve Race

SWC-114

MEDIUM

A spender observing a pending allowance update in the public mempool can front-run it: spending the old allowance, then immediately capturing the newly approved amount.

#VULNERABLE

```
token.approve(spender, newAmount);
```

OpenZeppelin SafeERC20 forceApprove

```
token.forceApprove(spender, newAmount);
```

Mitigation: Always reset allowances to zero before re-approving, use forceApprove, or adopt gasless EIP-2612 permit signatures.

26 · Missing Function Parameter Validation

SWC-123

MEDIUM

A function accepts an input, it can be an address, an amount, an array length, without checking it's sane, letting a zero address, zero amount, or malformed input slip through and corrupt state.

#VULNERABLE

```
function withdraw(address to, uint amount) external {
    balances[msg.sender] -= amount;
    payable(to).transfer(amount);
}
```

validate every external input explicitly

```
function withdraw(address to, uint amount) external {
    require(to != address(0), "zero address");
    require(amount > 0, "zero amount");
    balances[msg.sender] -= amount;
    payable(to).transfer(amount);
}
```

Mitigation: Validate every externally-supplied parameter explicitly.

04

Securing a Smart Contract

A repeatable workflow, the tools that support it, and the human side of security.

4 Securing a Smart Contract

Each layer here catches a different class of problem more cheaply than the next. Skipping the cheap layers just means the expensive ones, or attackers, find the bugs.

4.1 The security workflow



Schema 11. Chronological Secure Development Lifecycle (SDLC)

- **Threat-model before writing code.** Define your trust assumptions explicitly. Document every role, boundary, and privilege before a single line of Solidity is compiled. Assume every external caller is malicious and every third-party input is poisoned.
- **Default to audited libraries.** Never roll your own crypto, token standards, math libraries, or access control models. Use battle-tested OpenZeppelin contracts to eliminate entire categories of implementation errors.
- **Bake architecture in early.** Design with Checks-Effects-Interactions (CEI), pull-over-push payments, and least-privilege access controls from day one.
- **Adversarial fuzzing and invariant testing.** Move beyond basic unit tests. Write stateful fuzz tests (using Foundry or Echidna) targeting core protocol invariants. Fork mainnet state and simulate real-world attacks.
- **Independent professional audits.** Never rely solely on automated linters or internal reviews. Secure an external, experienced security review to catch subtle economic and architectural flaws before real capital touches your contracts.

4.2 Wallet & key hygiene

No amount of contract-level security matters if the keys controlling it are compromised. Most large losses in 2026 traced back to key and signer compromise, not code bugs — this section is about closing that gap.

- **Hardware wallets for anything that matters.** Private keys should never touch an internet-connected device. A hardware wallet signs transactions offline, so malware on your computer can't exfiltrate the key itself.
- **Never share your seed phrase, with anyone, ever!** No legitimate support team, exchange, or protocol will ever ask for it. Anyone who does is running a scam, full stop.
- **Multisig for anything holding real value.** A single key is a single point of failure. A multisig requires several independent signers to approve a transaction, so one compromised key isn't enough.
- **Verify every contract address before signing.** Phishing sites clone real interfaces pixel-for-pixel. Copy the contract address from the official docs or a trusted source, not from a link in a message or comment.
- **Read what you're signing.** A wallet approval isn't just "yes", it can grant unlimited spending rights over a token. Check the function being called and the amount before approving, especially on unfamiliar dApps.
- **Separate hot and cold wallets by purpose.** Keep a small "hot" wallet for everyday interactions and a cold, hardware-secured wallet for anything you're not actively using, so a compromised hot wallet never touches your real holdings.
- **Recognize common social engineering & phishing patterns:** Stay vigilant against manufactured urgency (e.g., "funds lost in 10 minutes"), fake support DMs, lookalike domains with subtle typos, malicious airdrop/giveaway prompts, and unauthorized signature requests designed to drain your tokens.

4.3 The tooling landscape

No single tool catches every class of bug, each one below is built to find a different kind of flaw, which is why real security processes stack several of them rather than picking just one.

Tool	Category	What it does
Slither	Static Analysis	Flags common vulnerability patterns fast by reading code without running it; ideal for CI.
Mythril	Symbolic Execution	Explores every reachable execution state to find paths an attacker could actually trigger, not just patterns that look risky.
Echidna	Fuzzing	Hammers your stated invariants (properties that must always hold) with thousands of random call sequences, looking for one that breaks them.
Foundry	Testing Framework	Fast Solidity-native testing with built-in fuzzing.
Certora	Formal Verification	Mathematically proves specified properties hold for <i>all</i> possible inputs, not just the ones tested.
Auditors	Human Review	Reasons about intent, business logic, and cross-contract economics—catches the complex logic exploits that no automated tool models.

4.4 The audit lifecycle



- **Scope:** Define exact boundaries, target contracts, commit hashes, and trust assumptions.
- **Review:** Auditors inspect code line by line, build threat models, and attempt exploits.
- **Report:** Findings are ranked by Likelihood × Impact and backed by step-by-step proofs.
- **Fix:** Developers patch vulnerabilities; dismissing a finding requires formal justification.
- **Re-review:** Auditors verify patches fix vulnerabilities without introducing regressions.

4.5 Severity Decision Matrix: Likelihood × Impact

Audit severities (Critical down to Informational) are calculated using a strict matrix combining probability and consequence.

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Informational

Severity Definitions:

- **Critical:** Instantly exploitable by anyone, leading to total fund drain or complete compromise.
- **High:** Severe impact under realistic conditions, or moderate impact that is trivially easy to trigger.
- **Medium:** Bounded risk limited funds at risk or requires narrow, unusual conditions.
- **Low / Info:** Gas optimizations, code quality issues, or theoretical edge cases with no path to exploitation.

Why this matters: Use this matrix to prioritize your remediation backlog (Criticals first) and frame auditor disputes around concrete likelihood and impact arguments rather than subjective disagreement.

05

Security Reviews & Audits

What actually happens when someone reviews your contract, and how to prepare for it.

5 Security Reviews & Audits

An external security review is not a substitute for rigorous internal testing; it is the final gatekeeper before mainnet. Understanding the standard audit lifecycle and preparing your codebase with a strict code freeze, comprehensive invariant documentation, and exhaustive fuzz tests ensures auditors spend their time hunting complex logical flaws rather than basic setup errors.

5.1 What is a security review?

A security review (commonly called an audit) is an independent examination of your contract's code, looking for bugs, design flaws, and ways the intended behavior could be broken or abused. It's performed by someone who didn't write the code, specifically so they bring no assumptions about what it's "supposed" to do — they read what it actually does.

5.2 How to prepare for a security review

The single biggest lever you have over how useful a review is: how ready your code is before the reviewers start the clock.

- **Clear, up-to-date documentation.** Explain what the protocol is supposed to do, the roles involved, and the trust assumptions; reviewers spend the first chunk of any engagement just building this mental model themselves if you don't hand it to them.
- **Freeze the code.** Reviewing a moving target wastes review time on re-reading changed code instead of finding bugs. Tag a specific commit and stop merging into it until the review concludes.
- **Code should be commented and readable.** NatSpec comments (`/// @notice`, `/// @dev`) on every public function, meaningful variable names, and small, single-purpose functions all directly speed up how fast a reviewer can build an accurate mental model.
- **Include your test suite, including fuzz tests.** A reviewer reading your tests learns what you were worried about and what you weren't; gaps in your own test coverage are often exactly where they'll look first.
- **Establish a communication channel with the reviewers.** A shared Discord/Slack channel or a scheduled check-in lets reviewers ask quick clarifying questions about intended behavior instead of guessing wrong either wastes their time or misses a real bug.

5.3 The 3 phases of a security review

A typical engagement runs in three distinct phases, not one long pass through the code.

5.3.1 Phase 1 — Initial review

- **Scoping:** the team and reviewers agree exactly what's in scope, which contracts, which commit, which parts of the system are assumed correct and not re-examined.
- **Reconnaissance:** reviewers read the code and documentation to build an independent understanding of what the protocol is supposed to do and who can do what.
- **Vulnerability identification:** reviewers actively try to break every assumption, using the tools, manual line-by-line reading, and adversarial thinking about each function.
- **Reporting:** findings are written up with severity, a concrete exploit scenario, and a recommended fix.

5.3.2 Phase 2 — Protocol fixes

- **Fixes issues:** the development team addresses each finding. Choosing not to fix something is a legitimate outcome too, but it should be a documented, explicit decision, not a silent skip.

- **Retests and adds tests:** every fix should come with a test that would have caught the original bug, so it can't silently regress later.

5.3.3 Phase 3 — Mitigation review

- **Reconnaissance:** reviewers re-examine specifically the code that changed, in the context of the original findings.
- **Vulnerability identification:** confirm each fix actually closes the issue, and check the fix itself didn't introduce a new one, which happens more often than teams expect.
- **Reporting:** a final report documenting which findings are resolved, which remain, and the state of the code at the end of the engagement.

5.4 Fuzz testing and formal verification, explained plainly

5.4.1 What is fuzz testing?

Instead of writing individual test cases with specific numbers you chose, a fuzzer generates thousands or millions of random, unexpected, and edge-case inputs and runs your code against all of them, looking for any input that breaks a rule you stated (an *invariant*) — for example, “the sum of all user balances should always equal the contract's total balance.” A human tester checks the cases they thought of; a fuzzer checks cases nobody thought of.

Tuning how hard the fuzzer tries: Foundry's fuzzer is configured in `foundry.toml`. Two settings matter most:

foundry.toml — fuzz configuration

```
[fuzz]
runs = 1000          # how many random inputs to try per test
seed = '0x2'         # fixes the "random" sequence -- reproducible runs
```

`runs` controls how many random inputs each fuzz test tries, higher catches more edge cases but costs more CI time; a fixed `seed` makes a run reproducible, which matters when you're trying to reproduce a failure someone else's CI found rather than chasing a new random sequence every time.

5.4.2 What is formal verification?

Formal verification goes a step further than fuzzing: instead of testing many inputs and hoping to find a counterexample, it mathematically proves a property holds for *every possible* input, with the same certainty as a mathematical proof. It's more expensive to set up and requires stating your properties in a formal specification language, but the guarantee it gives is categorically stronger than any amount of testing.

5.5 Bug bounty programs and competitive audits

Traditional single-firm audits aren't the only way to get eyes on a contract.

- **Bug bounty programs** pay independent researchers, ongoing, for responsibly disclosing vulnerabilities — usually scaled by severity, sometimes into the millions for critical findings on high-value protocols.
- **Competitive audits** (the model used by platforms like Code4rena and Sherlock) open a contract to many independent researchers at once, over a fixed window, competing to find the most valid, highest-severity issues. This trades the single reviewer's deep continuity for a much larger number of independent eyes and attack angles in parallel.

5.5.1 Study real incidents

Reading real exploit post-mortems is one of the best ways to learn. [rekt.news](#) provides detailed analyses of major exploits, while [Solodit](#) collects findings from public audit reports. Crytic's [Properties](#) repository also provides reusable invariant properties for fuzz testing.

06

Incident Response

A repeatable workflow, the tools that support it, and the human side of security.

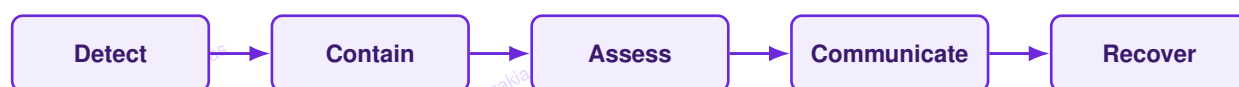
6 Incident Response

The protocols that survive an exploit are the ones that planned for it before it happened. Improvising a response during an active drain costs the minutes that determine how much is left to save.

6.1 Before: build the response before you need it

- **A tested pause switch, reachable by multisig.** Section 1 covered why `selfdestruct` is the wrong kill switch, a pausable pattern (OpenZeppelin's `Pausable`) is the right one, and it needs to actually be tested, not just written.
- **On-chain monitoring for anomalies.** Large withdrawals, unusual call patterns, or price deviations should trigger an alert before a human happens to notice, not after.
- **A written, rehearsed runbook.** Who has the authority to pause. Who they contact. What gets communicated, and to whom, in what order. Deciding this during a live incident wastes the time that matters most.
- **A funded, live bug bounty.** Giving a white hat a legitimate, well-paid path to report a bug is cheaper than the alternative — them selling the exploit to someone who won't.

6.2 During: the response sequence



Schema 12. The incident response sequence — each stage should already have an owner named in the runbook.

- **Detect:** monitoring or a report surfaces the incident.
- **Contain:** pause the contract or apply rate limits immediately (stopping the bleeding matters more than fully understanding the cause in the first minutes).
- **Assess:** once contained, determine what happened, what was lost, and whether the exploit is still exploitable elsewhere in the system.
- **Communicate:** tell users what happened, honestly and as early as reasonably possible.
- **Recover:** patch the root cause, verify the fix under audit, and only then resume normal operation.

6.3 After: the post-mortem

Publish an honest, technical account of what happened, why, and what changes as a result. Transparency after an incident is what rebuilds the trust the exploit destroyed.

07

Pre-Deployment Checklist

Final checks before deployment.

7 Pre-Deployment Checklist

Verify every item before deploying to mainnet.

■ PRINT-READY OFFICIAL SIGN-OFF SHEET ■

Physical verification required. Initial each section before deployment approval.

CODE CORRECTNESS & LOGIC

- ☐ State changes strictly follow Checks-Effects-Interactions (CEI).
- ☐ Reentrancy guards implemented on all value-moving entrypoints.
- ☐ Compiled with Solidity ≥ 0.8 ; every unchecked block explicitly justified.
- ☐ Low-level call return values verified; all token transfers wrapped via SafeERC20.
- ☐ All dynamic loops strictly bounded and safe against gas exhaustion DoS.
- ☐ No precision loss or truncation errors from integer division ordering.
- ☐ Every function's visibility is deliberately chosen (public/external/internal/private) — no helper left reachable by default.

ACCESS CONTROL & PRIVILEGES

- ☐ Every privileged function is strictly access-controlled; zero reliance on `tx.origin`.
- ☐ Administrative keys transitioned to multisig; governance changes timelocked.
- ☐ Initialization logic protected; disabled directly on implementation contracts.
- ☐ No unverified `delegatecall` targets allowed anywhere in the call graph.
- ☐ Storage layouts meticulously verified and aligned across all upgradeable proxies.
- ☐ Token supply/mint functions restricted to authorized roles only — no EOA can mint or alter supply without an explicit, checked permission.

ECONOMIC & EXTERNAL RISK

- ☐ Asset pricing sourced from manipulation-resistant oracles with strict heartbeat/staleness checks.
- ☐ All decentralized exchange swaps enforce explicit slippage limits and transaction deadlines.
- ☐ Zero reliance on predictable block variables for ran-

domness; Chainlink VRF (request/reveal split across transactions) integrated where required.

- ☐ Signed messages strictly bound via EIP-712 to unique nonces and explicit chain IDs.
- ☐ Vault implementations protected against first-depositor share inflation and donation attacks.
- ☐ Restaking protocols enforce rigid exposure caps and continuous operator health checks.
- ☐ No strict equality (`==`) against `address(this).balance` anywhere in the contract.
- ☐ No payable function loops and `delegatecalls` into other payable functions.
- ☐ Every integrated ERC20 checked against known non-standard behavior (fee-on-transfer, rebasing, ERC-777 hooks); balances measured by delta, not by trusting the transfer amount.
- ☐ Protocol-wide invariants (e.g., “no account is ever left insolvent,” $x \cdot y = k$) explicitly written down and covered by an invariant/fuzz test — not just checked locally inside each individual function.

PROCESS & OPERATIONS

- ☐ High test coverage achieved; stateful invariants extensively fuzzed; static analysis clean.
- ☐ Independent professional security audit completed; all findings formally resolved.
- ☐ Emergency pause switch, circuit-breaker mechanisms, and velocity limits fully tested on testnet.
- ☐ On-chain monitoring alerts, incident response runbooks, and bug bounty programs active.
- ☐ Cross-chain bridge payloads validated against decentralized quorums and strict cryptographic nonces.
- ☐ Compiler pragma pinned to an exact version; no floating $\wedge$ ranges.
- ☐ All magic numbers replaced with named constants; unchanging variables marked `immutable/constant`.
- ☐ Dependency versions pinned exactly; no floating ranges on imported libraries.

8 Glossary & Resources

Terms used in this guide:

Reentrancy re-entering a function via an external call before its state is finalized.

Flash loan uncollateralized borrowing repaid within the same transaction.

MEV profit from reordering, inserting, or censoring transactions.

Oracle a mechanism that brings off-chain data on-chain.

TWAP a manipulation-resistant time-weighted average price.

delegatecall run another contract's code against your own storage.

Proxy a contract that forwards logic to a swappable implementation.

Invariant a property that must always hold; a fuzzing/verification target.

Timelock a mandatory delay between scheduling and executing an action.

EIP-712 a standard for signing structured data bound to a domain.

AVS Actively Validated Service; off-chain infrastructure secured by restaked capital.

Slashing protocol-enforced destruction of staked collateral due to malicious behavior or operator failure.

ERC-4626 tokenized vault standard defining standardized share and asset mechanics.

Front-running observing a pending transaction in the mempool and paying a higher gas fee to execute ahead of it.

Symbolic Execution automated analysis that explores all mathematical paths through a program to find rule violations.

Formal Verification mathematical proof asserting that code satisfies specific formal properties across all inputs.

Multisig a wallet requiring multiple independent cryptographic signatures to authorize transactions.

Sanity Bounds hard-coded threshold limits preventing acceptance of anomalous or corrupted external input data.

Gas Exhaustion a denial-of-service condition where a transaction runs out of gas during unconstrained loop execution.

Storage Collision accidental overwriting of proxy state variables due to misaligned storage layouts between implementations.

Where to go deeper: SWC Registry · OpenZeppelin Contracts & Docs · OWASP Smart Contract Top 10 · Solidity Security Considerations · Ethereum.org Security · Public Audit Reports (Sherlock, Spearbit, Trail of Bits) · CTF Wargames (Ethernaut, Damn Vulnerable DeFi).

Security is a habit, not a milestone

The contracts that survive are the ones written defensively, reviewed independently, and monitored continuously. Assume you'll be attacked, design so the attack fails cheaply, and keep the code simple enough that a tired reviewer can still understand exactly what it does.

What's next: If this guide helped, the next step is applying it to your own contracts before they touch mainnet — not after.

You can see this framework applied to real codebases in my published audit reports at github.com/zakiabu78/my-audit-reports, or reach out if you want a second set of eyes on a specific contract, or want the checklist in a format you can actually run through with your team.